

Computer Vision Applications using OpenCV and YOLO

INTERNSHIP REPORT

Submitted by

**R. GANESH BABU
(25PDS517)**

In partial fulfillment of the requirement for
the award of the degree of
Master of Science in Data Science

Under the guidance of

Dr. A. NATRAJAN, M.C.A, M.Tech., Ph. D.,
Assistant Professor of Data Science



**DEPARTMENT OF DATA SCIENCE
ST. XAVIER'S COLLEGE (AUTONOMOUS)**

(Recognized as 'College with Potential for Excellence' by UGC)

(Accredited by NAAC at A++ Grade with a CGPA of 3.66 out of 4 in IV Cycle)

PALAYAMKOTTAI –627002

June 2026

DEPARTMENT OF DATA SCIENCE

ST. XAVIER'S COLLEGE (AUTONOMOUS)

(Recognized as 'College with Potential for Excellence' by UGC)

(Accredited by NAAC at A++ Grade with a CGPA of 3.66 out of 4 in IV Cycle)

PALAYAMKOTTAI –627002



BONAFIDE CERTIFICATE

This is to certify that the internship work entitled “**Computer Vision Applications using Python, OpenCV, and YOLO**” is the bonafide work of **R. GANESH BABU (25PDS517)** who carried out the internship under my supervision and submitted during the academic year 2026-2027.

GUIDE

HEAD OF THE DEPARTMENT

The Viva-Voce held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I **R. Ganesh Babu**, a student enrolled in **Department of Data Science** at St. Xavier's College (Autonomous), Palayamkottai, hereby declare that the documentation presented for my internship work, titled "**Computer Vision Applications using Python, OpenCV, and YOLO**", is entirely my original work. I assert that this has been done under the supervision and guidance of **Dr. A. Natrajan** and in accordance with the rules and regulations set forth by **Department of Data Science** for the completion of the internship.

Place : Palayamkottai

Date :

Student's Signature

Internship Offer Letter

ANDUM TECHNOLOGIES PRIVATE LIMITED

12/660, B2, F3, PATHANJALI GARDENS, 7TH STREET, VEERABADRA NAGAR, MEDAVAKKAM,
CHENNAI, TAMILNADU, INDIA, 600100



Date : 10 May 2026

To

R. Ganesh Babu
College Name: St. Xavier's College (Autonomous),
Palayamkottai

Dear R. Ganesh Babu,

We are pleased to offer you the Data Science Internship position at Andum Technologies Private Limited. We are impressed with your educational background and skills therefore believe that you will be able to contribute to our Data Science project as an Intern.

Your internship will commence on 14 May 2026 and will conclude on 14 Jun 2026. During this period, you will be engaged in a project focused on Claude code and Computer vision, where you will have the opportunity to apply your data science skills and contribute to our data-driven decision-making process.

We look forward to your acceptance and to working with you.

Sincerely,

Candidate

A handwritten signature in blue ink, appearing to read 'Mun'.

Director

Internship Work Diary

S. No	Date	Nature of Work	Signature of Authority
1.	19-05-2026	Introduction to Computer Vision	
2.	20-05-2026	Overview of OpenCV Libraries	
3.	21-05-2026	Video Frame Extraction from CCTV Footage	
4.	22-05-2026	Image Skewing and Flipping using OpenCV	
5.	23-05-2026	Image Rotation and Grayscale Conversion using OpenCV	
6.	25-05-2026	Edge Detection using OpenCV	
7.	26-05-2026	Face Detection using OpenCV	
8.	27-05-2026	Face Recognition under Different Lighting Conditions	
9.	28-05-2026	CCTV Boundary Detection	
10.	29-05-2026	Person Tracking in CCTV Monitoring	
11.	30-05-2026	Privacy-Based Monitoring using Boundary Region Selection	
12.	01-05-2026	Introduction to YOLO Object Detection Model	
13.	02-06-2026	Face Detection using YOLO	

14.	03-06-2026	Understanding Haar Cascade, DNN, and MTCNN Models	
15.	04-06-2026	Geo-Fencing using ROI and State Management	
16.	05-06-2026	YOLO Object Detection with ByteTrack	
17.	06-06-2026	YOLO Object Tracking using DeepSORT	
18.	08-06-2026	Introduction to Physical AI	
19.	09-06-2026	Student Attendance and Monitoring System	
20.	10-06-2026	Multi-Object Detection and Tracking using YOLO	
21.	11-06-2026	Real-Time Surveillance Analytics using Computer Vision	
22.	12-06-2026	Human Activity Detection and Monitoring	
23.	13-06-2026	Smart CCTV Monitoring and Event Detection	
24.	15-06-2026	Integration of Computer Vision Models	
25.	16-06-2026	Final Computer Vision Project Development and Presentation	

ACKNOWLEDGEMENT

With humble, wholehearted, and deep sense of gratitude, I thank **God, the Almighty** for His immense blessings and graces in helping me complete this piece of work.

I am profoundly grateful to **Rev. Fr. Dr. Godwin Rufus, S.J., Principal**, and **Dr. A. Lourdusamy, Deputy Principal** for giving me the opportunity to pursue my studies and utilize the facilities available at this esteemed institution.

I gracefully remember and express my immense gratitude to **Dr. A. Natrajan, M.C.A., M. Tech., Ph.D.**, Head, Department of Data Science, St. Xavier's College (Autonomous), Palayamkottai, for his kind patronage and assistance.

I express my heartfelt thanks to **Dr. A. Natrajan, M.C.A., M. Tech., Ph.D.**, Assistant Professor, Department of Data Science, St. Xavier's College (Autonomous), Palayamkottai, for his keen interest, and constant encouragement throughout the course of this work. His support was instrumental in the successful completion of my internship.

I would also like to thank **all the faculty members and technical staff of the Department of Data Science** for their assistance and cooperation during the different stages of the internship. Last but not least, I extend my warmest thanks to **my beloved parents** for their unwavering love, prayers, and moral support throughout this journey.

I would like to express my sincere gratitude to my mentor, **Mr. Suganth Mani**, for his constant guidance, valuable suggestions, encouragement, and unwavering support throughout the internship. His expertise, patience, and insightful feedback played a significant role in enhancing my learning experience and successfully completing this internship. I am truly grateful for his mentorship and the knowledge he shared during this journey.

R.GANESH BABU
25PDS517

Computer Vision Applications using OpenCV and YOLO

ABSTRACT

The field of **Computer Vision** plays a significant role in modern artificial intelligence by enabling machines to interpret and understand visual information from images and videos. This internship work was designed to provide practical exposure to computer vision concepts using **Python, OpenCV, and YOLO**, while developing a strong foundation in image processing, object detection, and real-time surveillance applications.

During the internship, I began by learning the fundamentals of Computer Vision and the OpenCV library, which included image manipulation techniques such as frame extraction, image skewing, flipping, rotation, grayscale conversion, and edge detection. These activities helped me understand how digital images are processed and prepared for computer vision tasks.

The next phase focused on advanced computer vision applications such as face detection, face recognition under different lighting conditions, CCTV boundary detection, person tracking, privacy-based monitoring using region selection, and geo-fencing using ROI and state management. These tasks provided hands-on experience in developing intelligent surveillance systems capable of monitoring people and activities in real time.

Overall, this internship provided a comprehensive learning experience in computer vision, covering image processing, object detection, object tracking, and intelligent monitoring systems. It strengthened my technical skills in Python, OpenCV, and YOLO while improving my problem-solving abilities and practical understanding of real-world AI-powered computer vision applications.

TABLE OF CONTENT

CHAPTER NO.	TITLE	PAGE NO.
1.	INTRODUCTION	
	1.1 About the Company	1
	1.2 Objectives	2
2.	THE ROLE OF COMPUTER VISION IN MODERN INDUSTRIES	
	2.1. Evolution of Computer Vision	4
	2.2. Applications of Computer Vision	4
	2.3. Types of Computer Vision	5
	2.4. Real world Examples of Computer Vision	6
	2.5. Benefits of computer vision	6
	2.6. Tools Used in Computer Vision	7
3.	OVERVIEW OF TOOLS AND TECHNOLOGIES	
	3.1. Open CV Library	8
	3.2. YOLO Object Detection Model	9
	3.3. Haar Cascade, DNN, MTCNN, ArcFace	9
	3.4. ByteTrack and DeepSORT	9
4.	OPENCV - IMAGE AND VIDEO PROCESSING	
	4.1. What is OpenCV?	10
	4.2. Features of OpenCV	10
	4.3. Application of OpenCV	11
	4.4. Video Frame Extraction	11
	4.5. Image Processing Using OpenCV	13
	4.6. Face and Edge detection	16
	4.7. Real-world Use Case Examples	17
	4.8. Role of OpenCV	17
	4.9. Advantages of OpenCV	17
5.	YOLO – OBJECT DETECTION AND TRACKING	
	5.1. Introduction	18
	5.2. Role of YOLO in Object Detection	18
	5.3. Key Features Utilized	18
	5.4. Object Detection and Tracking using YOLO	18
	5.5. Summary	21
6.	HAAR CASCADE, DNN, MTCNN, AND ARCFACE – FACE DETECTION AND RECOGNITION	
	6.1. Introduction	22
	6.2. Role of Face Detection Models in Computer Vision	22
	6.3. Comparison of Haar Cascade, DNN, MTCNN, and ArcFace	24
	6.4. Applications in Face Detection and Recognition	24
7.	PROJECT WORK – STUDENT ATTENDANCE AND MONITORING SYSTEM	25
8.	RESULTS AND OBSERVATIONS	28
9.	CHALLENGES AND SOLUTIONS	29
10.	KEY LEARNINGS AND REFLECTIONS	30
11.	CONCLUSION	31
	APPENDIX	

1. INTRODUCTION

Computer Vision is a branch of Artificial Intelligence (AI) that enables computers to interpret and understand images and videos. It combines image processing, machine learning, and deep learning techniques to identify objects, recognize faces, track movements, and analyze visual information automatically. Today, computer vision is widely used in surveillance systems, healthcare, autonomous vehicles, industrial automation, and smart city applications.

During this internship, I gained practical experience in Computer Vision using **Python, OpenCV, and YOLO**. I learned various image processing techniques such as video frame extraction, image skewing, image flipping, image rotation, grayscale conversion, edge detection, face detection, face recognition, CCTV boundary monitoring, person tracking, privacy-based monitoring using Region of Interest (ROI), geo-fencing, and object tracking using ByteTrack and DeepSORT.

The internship concluded with the development of a **Face Tracking Attendance System**, which integrates YOLO-based person detection, face recognition, geo-fencing, and automated attendance logging. This internship enhanced my technical knowledge and practical skills in Computer Vision while providing hands-on experience in developing real-time intelligent surveillance and monitoring applications.

Overall, this internship strengthened my understanding of computer vision concepts and provided valuable experience in developing intelligent vision-based systems capable of solving real-world surveillance and monitoring challenges.

1.1. About the Company

Andum Technology is a software development company located in **Chennai, Tamil Nadu**, specializing in delivering innovative, reliable, and customized software solutions for businesses across various industries. The company is committed to helping organizations achieve digital transformation by developing high-quality applications using modern technologies and industry best practices.

The company offers a wide range of services, including web application development, mobile application development, desktop application development, cloud-based solutions, and custom software development. Its team of experienced software developers, designers, testers,

and project managers work collaboratively to deliver scalable, secure, and user-friendly software solutions that meet the specific requirements of clients.

Andum Technology follows a structured software development lifecycle consisting of requirement analysis, system design, development, testing, deployment, and maintenance. By adopting agile methodologies and industry-standard practices, the company ensures timely project delivery while maintaining high standards of quality, security, and performance. Each project is developed with a strong focus on customer satisfaction, continuous improvement, and technical excellence.

The company places great emphasis on quality assurance by conducting comprehensive testing at every stage of the development process. This ensures that the software products are reliable, efficient, and capable of meeting both functional and non-functional requirements. Andum Technology also stays updated with the latest technologies and continuously enhances its development practices to provide innovative and future-ready solutions.

To meet the diverse needs of its clients, Andum Technology offers flexible engagement models such as Dedicated Team, Fixed-Price Projects, and Time-and-Material contracts. These models enable businesses to choose the most suitable approach based on their project scope, budget, and timeline while ensuring transparency, effective communication, and successful project execution.

With its commitment to innovation, customer satisfaction, and high-quality software development, Andum Technology has established itself as a trusted technology partner for organizations seeking reliable and efficient digital solutions.

1.2 Objectives

The primary goal of this internship was to gain practical knowledge and hands-on experience in the field of **Computer Vision** through image processing, object detection, and real-time surveillance applications. The following were the key objectives:

1. Understand the Fundamentals of Computer Vision

To learn the basic concepts of Computer Vision and understand how images and videos are processed for intelligent decision-making using Artificial Intelligence.

2. Gain Proficiency in OpenCV

To develop practical skills using OpenCV for:

- Image processing and manipulation
- Video frame extraction

- Image transformation techniques
- Edge detection and grayscale conversion

3. Learn Image Processing Techniques

To perform various image processing operations, including:

- Image skewing, flipping, and rotation
- Grayscale image conversion
- Edge detection using OpenCV
- Image enhancement for computer vision applications

4. Develop Face Detection and Recognition Systems

To gain experience in:

- Face detection using OpenCV and Haar Cascade
- Face recognition under different lighting conditions
- Understanding DNN, MTCNN, and YOLO-based face detection models
- Comparing different face detection techniques

5. Explore Object Detection and Tracking using YOLO

- To understand the working of the YOLO (You Only Look Once) model for real-time object detection.
- To implement person detection and tracking using ByteTrack and DeepSORT.
- To apply object tracking techniques in CCTV surveillance applications.

6. Implement Intelligent Surveillance Applications

- To develop CCTV monitoring systems using Region of Interest (ROI) and Geo-Fencing.
- To implement privacy-based monitoring through boundary region selection.
- To understand event detection and real-time monitoring in surveillance systems.

7. Adapt to a Professional Work Environment

- Working on structured internship tasks improved my technical, analytical, and problem-solving skills.
- I learned to work collaboratively, follow project timelines, and apply industry-standard development practices while building real-world Computer Vision applications.

2. THE ROLE OF COMPUTER VISION IN MODERN INDUSTRIES

2.1 Evolution of Computer Vision

Computer Vision has evolved significantly over the years with advancements in Artificial Intelligence and Deep Learning.

1. Traditional Image Processing – Used basic techniques such as filtering, thresholding, and edge detection.
2. Feature-Based Methods – Introduced algorithms like Haar Cascade for detecting faces and objects.
3. Machine Learning Era – Enabled computers to classify images using extracted features.
4. Deep Learning Era – Convolutional Neural Networks (CNNs) improved the accuracy of image recognition.
5. Modern Computer Vision – Models like YOLO, MTCNN, and ArcFace provide real-time object detection, face recognition, and intelligent surveillance.

2.2. Applications of Computer Vision

Computer Vision is widely used in various industries to automate visual tasks and improve decision-making.

1. Healthcare

- Assists doctors in detecting diseases from X-rays, MRI, and CT scan images.
- Supports medical diagnosis and treatment planning with high accuracy.

2. Security and Surveillance

- Enables face recognition, person tracking, and intrusion detection in CCTV systems.
- Improves public safety through real-time monitoring and alert generation.

3. Autonomous Vehicles

- Detects roads, traffic signs, pedestrians, and other vehicles.
- Helps self-driving cars navigate safely and avoid obstacles.

4. Manufacturing and Quality Inspection

- Identifies defective products on production lines automatically.
- Improves product quality while reducing manual inspection efforts.

5. Retail and E-commerce

- Tracks customer behavior and shopping patterns.
- Supports automated checkout and inventory management.

6. Agriculture

- Monitors crop health and detects plant diseases using drone images.
- Assists farmers in improving crop yield through precision farming.

7. Education and Smart Attendance Systems

- Automates student attendance using face recognition technology.
- Monitors classrooms to improve safety and classroom management.

8. Robotics and Automation

- Helps robots recognize objects and navigate their surroundings.
- Enables automated pick-and-place operations in industries.

2.3. Types of Computer Vision

1. Image Classification

- Identifies and classifies an image into a predefined category.
- Used in applications such as disease detection, product recognition, and wildlife monitoring.

2. Object Detection

- Detects and locates one or more objects within an image or video.
- Commonly used in CCTV surveillance, traffic monitoring, and autonomous vehicles.

3. Face Detection and Recognition

- Detects human faces and identifies individuals based on facial features.
- Used in smart attendance systems, mobile phone face unlock, and access control.

4. Image Segmentation

- Divides an image into multiple meaningful regions or objects.
- Widely used in medical imaging, satellite image analysis, and autonomous driving.

5. Object Tracking

- Tracks the movement of detected objects across video frames.
- Used in CCTV monitoring, sports analytics, and intelligent surveillance systems.

6. Optical Character Recognition (OCR)

- Extracts and converts text from images or scanned documents into editable text.
- Used in document digitization, passport verification, and number plate recognition.

7. Pose Estimation

- Detects and tracks the position of the human body and its key joints.
- Used in fitness tracking, gesture recognition, and human activity analysis.

2.4. Real-World Examples of Computer Vision

- Face Unlock in Smartphones
- CCTV Person Detection and Tracking
- Automatic Number Plate Recognition (ANPR)
- Self-Driving Cars
- Medical Image Diagnosis
- Smart Attendance Systems
- Industrial Quality Inspection

2.5. Benefits of Computer Vision

1. Improved Accuracy

- Computer Vision reduces human errors by performing visual tasks with high precision.
- It provides reliable results in applications such as face recognition and object detection.

2. Automation of Tasks

- Automates repetitive tasks such as image inspection and attendance monitoring.
- Reduces manual effort and saves time.

3. Real-Time Monitoring

- Enables continuous monitoring through CCTV and surveillance systems.
- Detects suspicious activities and sends instant alerts.

4. Enhanced Security

- Supports face recognition, access control, and intrusion detection.
- Improves safety in public places, offices, and institutions.

5. Increased Productivity

- Speeds up industrial inspection and manufacturing processes.
- Helps organizations complete tasks more efficiently.

6. Better Decision-Making

- Extracts meaningful information from images and videos.
- Assists businesses and organizations in making informed decisions.

7. Cost Efficiency

- Reduces operational costs by minimizing manual inspection.
- Optimizes resources through intelligent automation.

2.6. Tools Used in Computer Vision

Computer Vision applications rely on various software tools, libraries, and frameworks to perform tasks such as image processing, object detection, face recognition, and object tracking. During the internship, several widely used Computer Vision tools and technologies were studied to understand their functionality and real-world applications.

- **Python:** Python is the primary programming language used for Computer Vision because of its simple syntax, extensive libraries, and strong community support.
- **OpenCV:** OpenCV (Open Source Computer Vision Library) is an open-source library that provides numerous functions for image processing, video analysis, object detection, and feature extraction.
- **YOLO (You Only Look Once):** YOLO is a real-time object detection algorithm that can identify and classify multiple objects in an image or video with high speed and accuracy.
- **Haar Cascade:** Haar Cascade is a machine learning-based object detection method commonly used for face, eye, and smile detection in images and live video streams.
- **OpenCV DNN:** The OpenCV Deep Neural Network (DNN) module enables the execution of pre-trained deep learning models for tasks such as image classification, object detection, and face recognition.
- **MTCNN:** Multi-task Cascaded Convolutional Neural Networks (MTCNN) is an advanced face detection framework that accurately detects facial landmarks and is widely used in face recognition systems.
- **ArcFace:** ArcFace is a deep learning-based face recognition model that generates highly accurate facial embeddings for identity verification and recognition.
- **ByteTrack:** ByteTrack is a modern multi-object tracking algorithm that efficiently tracks multiple moving objects in video sequences while maintaining high accuracy.
- **DeepSORT:** DeepSORT (Deep Simple Online and Realtime Tracking) combines object detection with deep learning-based appearance features to provide reliable object tracking across video frames.

These tools play a significant role in developing intelligent Computer Vision applications for security, surveillance, healthcare, autonomous vehicles, industrial automation, and many other real-world domains.

3. OVERVIEW OF TOOLS AND TECHNOLOGIES

During the course of my internship, I worked extensively with several advanced computer vision tools and deep learning technologies, including OpenCV, YOLO, Haar Cascade, DNN, MTCNN, ArcFace, ByteTrack, and DeepSORT. Each tool played a specific role in image processing, object detection, face recognition, and multi-object tracking. These technologies provided practical exposure to developing intelligent surveillance systems and real-time computer vision applications.

3.1. OpenCV Library

OpenCV (Open Source Computer Vision Library) served as the primary tool for image and video processing throughout the internship. It was used to perform various image processing operations such as video frame extraction, image skewing, image flipping, image rotation, grayscale conversion, and edge detection. OpenCV also enabled real-time face detection, image enhancement, and Region of Interest (ROI) selection for surveillance applications. Through these practical tasks, I gained a strong understanding of digital image processing and computer vision fundamentals.

- **Image Filtering:** OpenCV was used to apply different image filters such as Gaussian blur, median filtering, and sharpening techniques to improve image quality and reduce unwanted noise.
- **Feature Detection:** OpenCV provides various feature detection algorithms like ORB (Oriented FAST and Rotated BRIEF) that help in identifying important patterns and key points in images.
- **Object Detection Support:** OpenCV supports integration with various pre-trained models and deep learning frameworks to perform object detection and classification tasks.
- **Video Processing:** OpenCV enables efficient processing of video streams by capturing frames, analyzing individual frames, and applying computer vision algorithms in real time.
- **Camera Calibration:** OpenCV provides tools for camera calibration, which helps in correcting lens distortion and improving accuracy in vision-based applications.
- **Machine Learning Integration:** OpenCV can be combined with machine learning techniques to develop intelligent systems for recognition, tracking, and automated decision-making.
- **Real-Time Application Development:** Due to its speed and optimized algorithms, OpenCV is widely used for real-time applications such as surveillance systems, autonomous vehicles, robotics, and smart monitoring solutions.

These features made OpenCV an essential tool for understanding and implementing various computer vision concepts during the internship.

3.2. YOLO Object Detection Model

YOLO (You Only Look Once) is a state-of-the-art deep learning model designed for real-time object detection. During the internship, YOLO was used to detect and identify people and faces in images and CCTV video streams. It provided high-speed and accurate detection by processing an entire image in a single pass. YOLO was also integrated with tracking algorithms to develop intelligent surveillance applications, including person tracking, geo-fencing, and the Face Tracking Attendance System.

3.3. Haar Cascade, DNN, MTCNN, and ArcFace

Multiple face detection and recognition models were studied and implemented to understand their working principles and performance. **Haar Cascade** was used as a traditional face detection algorithm, while **OpenCV DNN** provided improved accuracy using deep neural networks. **MTCNN** was used for detecting faces and facial landmarks under different conditions, and **ArcFace** was introduced for highly accurate face recognition using deep learning embeddings. Comparing these models helped understand their advantages, limitations, and real-world applications in security and surveillance systems.

3.4. ByteTrack and DeepSORT

ByteTrack and DeepSORT are advanced multi-object tracking algorithms used to maintain the identity of detected objects across consecutive video frames. During the internship, these algorithms were integrated with YOLO to perform real-time person tracking in CCTV surveillance. They were also used for geo-fencing, boundary monitoring, and automated attendance systems. Their ability to track multiple people accurately in dynamic environments made them essential components of modern intelligent surveillance applications.

They helped in applications such as geo-fencing, boundary monitoring, and automated attendance systems by accurately tracking multiple individuals. Their efficient tracking capability in dynamic environments made them important components of modern intelligent surveillance applications.

4. OPENCV – IMAGE AND VIDEO PROCESSING

As part of the Computer Vision internship, I extensively used the **Open Source Computer Vision Library (OpenCV)** to perform various image and video processing tasks. OpenCV served as the primary tool for developing computer vision applications, enabling operations such as image transformation, video analysis, feature extraction, face detection, and edge detection. Throughout the internship, OpenCV helped me understand how digital images and videos can be processed efficiently to build intelligent real-time surveillance systems.

4.1. What is OpenCV?

OpenCV (Open Source Computer Vision Library) is an open-source library designed for Computer Vision, Machine Learning, and Image Processing applications.

It was originally developed by Intel and is currently maintained by the OpenCV community. OpenCV supports multiple programming languages including Python, C++, Java, and MATLAB, making it one of the most widely used computer vision libraries.

4.2. Features of OpenCV

1. Image and Video Processing

- OpenCV provides powerful functions for processing both images and videos.
- It supports operations such as filtering, resizing, rotation, and color conversion.

2. Real-Time Computer Vision

- OpenCV enables real-time face detection, object detection, and motion analysis.
- It is widely used in surveillance systems and AI-based vision applications.

3. Cross-Platform Support

- OpenCV runs on Windows, Linux, macOS, and Android platforms.
- It supports programming languages such as Python, C++, and Java.

4. Open-Source Library

- OpenCV is free to use for educational, research, and commercial projects.
- It has a large developer community and extensive documentation.

5. Deep Learning Integration

- OpenCV can be integrated with deep learning frameworks like TensorFlow and PyTorch.
- It supports advanced AI models such as YOLO for object detection.

6. High Performance

- OpenCV is optimized for fast image and video processing.
- It can efficiently process high-resolution images and live video streams.

4.3. Applications of OpenCV

1. Face Detection and Recognition

- OpenCV detects and recognizes human faces from images and videos.
- It is widely used in attendance systems and security applications.

2. CCTV Surveillance

- OpenCV processes live CCTV video to detect and track people.
- It helps improve security through real-time monitoring.

3. Medical Image Analysis

- It assists in analyzing X-rays, MRI, and CT scan images.
- It supports doctors in disease detection and diagnosis.

4. Autonomous Vehicles

- OpenCV detects roads, traffic signs, pedestrians, and obstacles.
- It enables safe navigation for self-driving vehicles.

5. Industrial Automation

- OpenCV performs automated quality inspection in manufacturing.
- It identifies product defects quickly and accurately.

6. Augmented Reality

- It is used to overlay digital objects onto real-world environments.
- It enhances interactive applications in gaming and education.

7. Robotics

- OpenCV helps robots identify objects and navigate their surroundings.
- It supports automation in warehouses and industries.

4.4. Video Frame Extraction

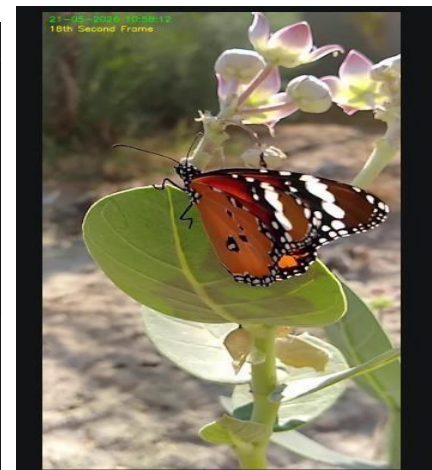
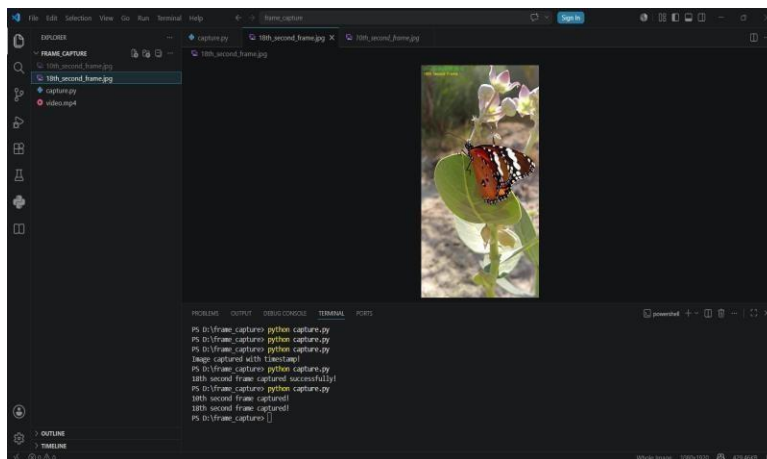
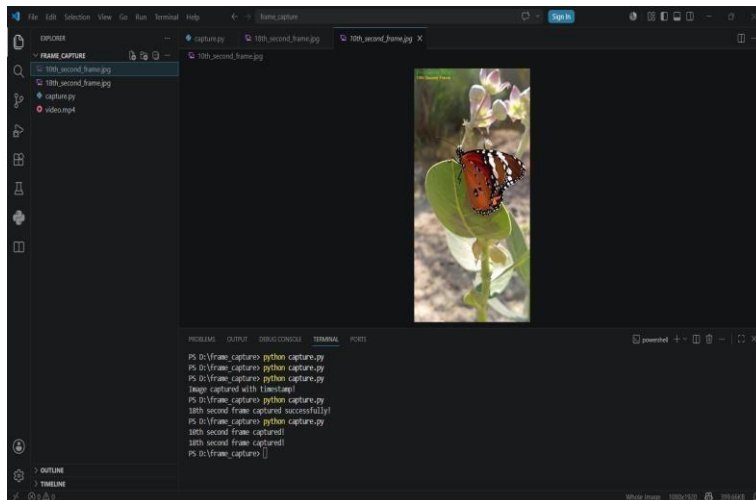
During the internship, OpenCV was used to extract individual frames from CCTV videos. Each frame was processed separately for object detection, face recognition, and tracking applications.

Frame extraction improves processing efficiency by converting continuous video streams into individual images that can be analyzed using Computer Vision algorithms.

CODING:

```
import cv2
from datetime import datetime
cap = cv2.VideoCapture("video.mp4")
if not cap.isOpened():
    print("Error opening video")
    exit()
fps = cap.get(cv2.CAP_PROP_FPS)
seconds_list = [10, 18]
for second in seconds_list:
    frame_number = int(fps * second)
    cap.set(cv2.CAP_PROP_POS_FRAMES, frame_number)
    success, frame = cap.read()
    if success:
        current_time = datetime.now().strftime("%d-%m-%Y %H:%M:%S")
        cv2.putText(
            frame,
            current_time,
            (20, 40),
            cv2.FONT_HERSHEY_SIMPLEX,
            1,
            (0, 255, 0),
            2
        )
        frame_text = f"{second}th Second Frame"
        cv2.putText(
            frame,
            frame_text,
            (20, 80),
            cv2.FONT_HERSHEY_SIMPLEX,
            1,
            (0, 255, 255),
            2
        )
        output_name = f"{second}th_second_frame.jpg"
        cv2.imwrite(output_name, frame)
        print(f"{second}th second frame captured!")
    else:
        print(f"Could not capture {second}th second frame")
cap.release()
```

OUTPUT:



4.5. Image Processing Using OpenCV

Several image processing techniques were implemented using OpenCV, including image skewing, image flipping, image rotation, grayscale conversion, and image enhancement. These operations improve image quality and prepare images for further analysis by deep learning models such as YOLO.

Image preprocessing plays a vital role in increasing the accuracy of object detection and face recognition systems.

Image Skewing - CODING:

```
import cv2
import numpy as np
image = cv2.imread("input.jpg")
h, w = image.shape[:2]
skew_factor = 0.3
M = np.float32([
```

```

[1, skew_factor, 0],
[0, 1, 0]
])
new_width = int(w + abs(skew_factor * h))
skewed = cv2.warpAffine(image, M, (new_width, h))
cv2.imwrite("skewed_output.jpg", skewed)
cv2.imshow("Skewed Image", skewed)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

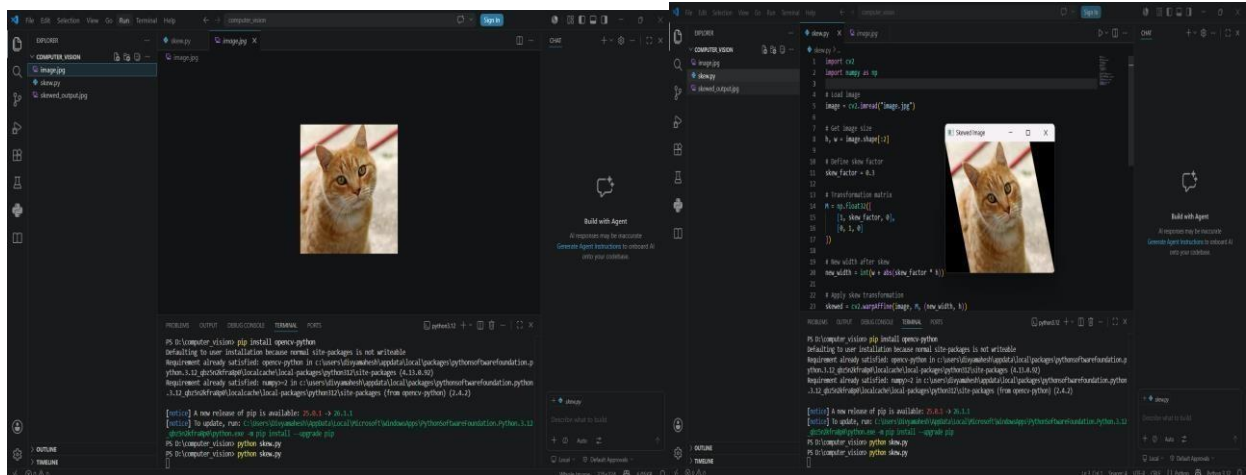


Image Flipping - CODING:

```

import cv2
image = cv2.imread("image.jpg")
flipped = cv2.flip(image, 1)
cv2.imshow("Original Image", image)
cv2.imshow("Flipped Image", flipped)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

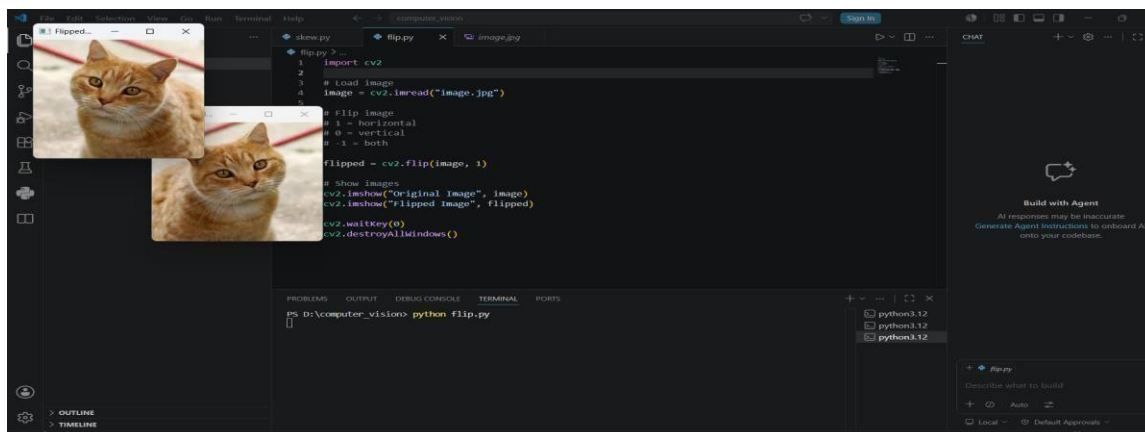


Image rotation - CODING:

```

import cv2
image = cv2.imread("cat.jpg")

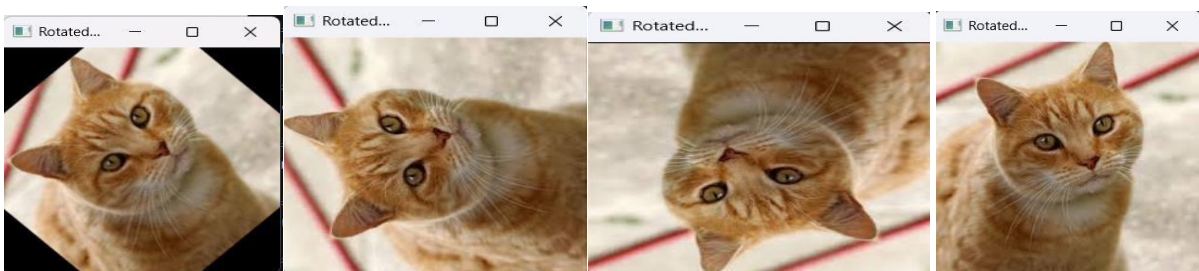
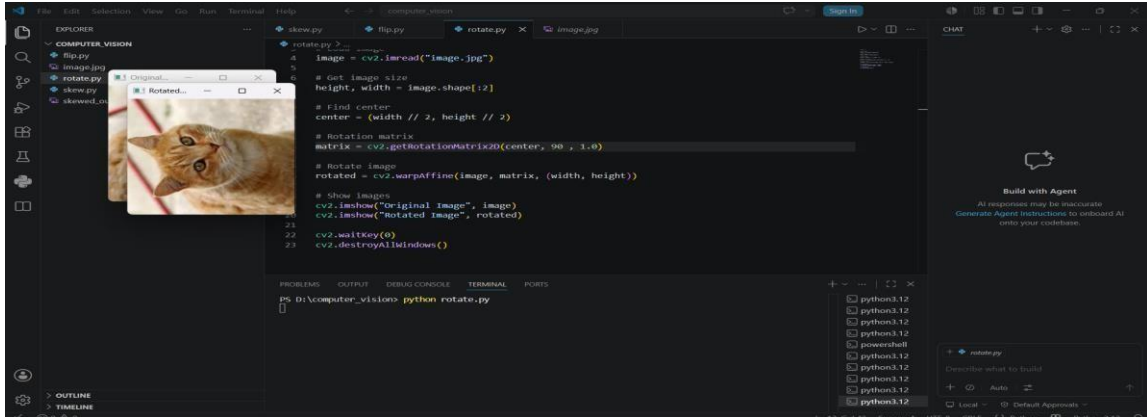
```



```

height, width = image.shape[:2]
center = (width // 2, height // 2)
matrix = cv2.getRotationMatrix2D(center, 45, 1.0)
rotated = cv2.warpAffine(image, matrix, (width, height))
cv2.imshow("Original Image", image)
cv2.imshow("Rotated Image", rotated)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

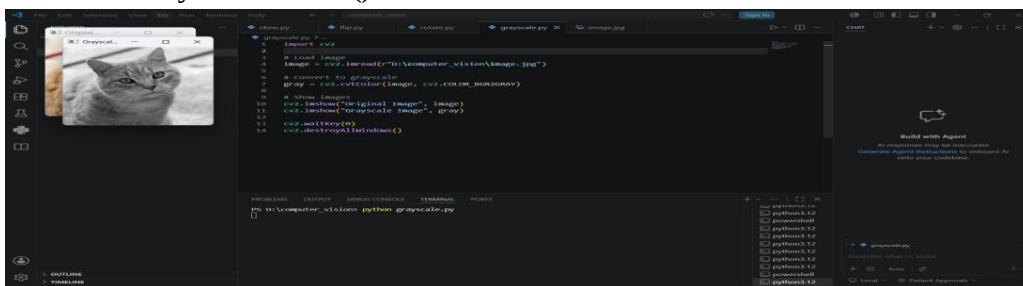


GrayScale Conversion - CODING:

```

import cv2
image = cv2.imread(r"D:\computer_vision\image.jpg")
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
cv2.imshow("Original Image", image)
cv2.imshow("Grayscale Image", gray)
cv2.waitKey(0)
cv2.destroyAllWindows()

```



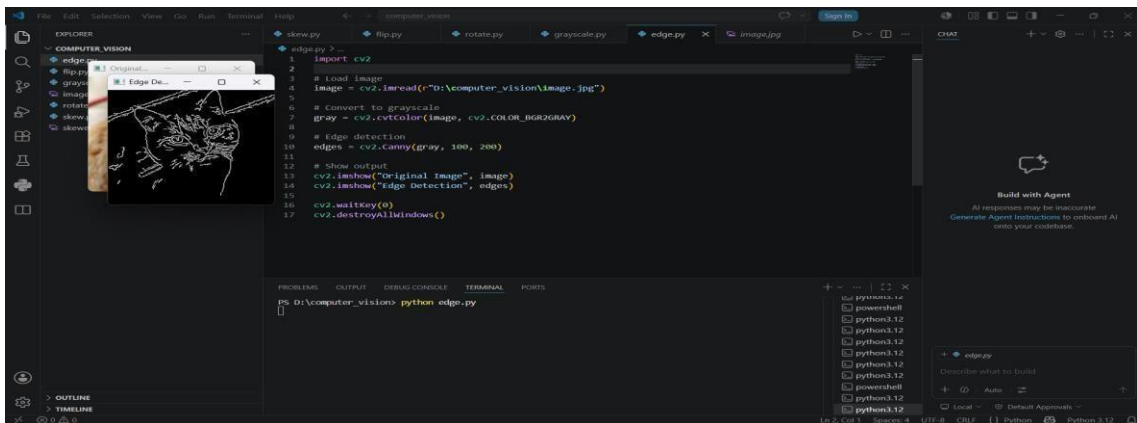
4.6. Face and Edge Detection

OpenCV provides efficient algorithms for detecting faces and image edges.

During the internship, Haar Cascade classifiers and OpenCV functions were used for face detection, while Canny Edge Detection was used to identify object boundaries within images. These techniques form the foundation for many Computer Vision applications, including surveillance, attendance systems, and intelligent monitoring.

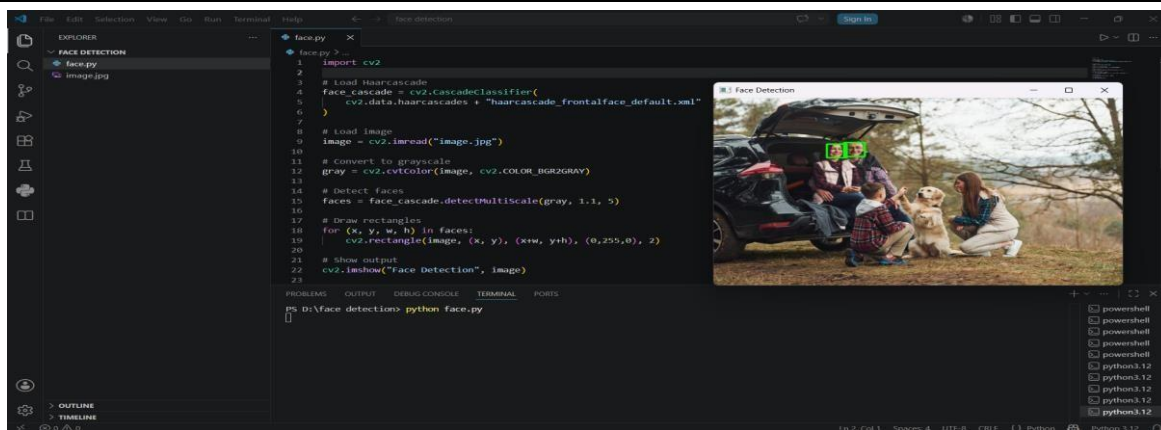
Edge detection - CODING:

```
import cv2
image = cv2.imread(r"D:\computer_vision\cat.jpg")
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 100, 200)
cv2.imshow("Original Image", image)
cv2.imshow("Edge Detection", edges)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Face Detection - CODING:

```
import cv2
image = cv2.imread("image.jpg")
if image is None:
    print("Image not found")
    exit()
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_frontalface_default.xml")
faces = face_cascade.detectMultiScale(gray, 1.3, 5)
print("Faces detected:", len(faces))
for (x, y, w, h) in faces:
    cv2.rectangle(image, (x, y), (x+w, y+h), (0, 255, 0), 2)
cv2.imshow("Face Detection", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



4.7. Real-World Use Case Examples

Use Case	Role of OpenCV
CCTV Surveillance	Detects and tracks people in real time
Face Recognition System	Detects and recognizes human faces
Smart Attendance System	Automates attendance using facial recognition
Medical Imaging	Enhances medical images for diagnosis
Autonomous Vehicles	Detects lanes, pedestrians, and traffic signs
Industrial Inspection	Detects product defects automatically

4.8. Role of OpenCV

1. Performs image preprocessing and enhancement.
2. Processes video streams for real-time analysis.
3. Detects faces, objects, and edges.
4. Supports feature extraction and image transformation.
5. Integrates with deep learning models such as YOLO.
6. Enables development of intelligent surveillance application.

4.9. Advantages of OpenCV

1. Free and open-source library.
2. Fast and efficient image processing.
3. Supports multiple programming languages.
4. Cross-platform compatibility.
5. Large community support and documentation.

5. YOLO – OBJECT DETECTION AND TRACKING

5.1 Introduction

YOLO (You Only Look Once) is a real-time object detection algorithm used in computer vision. It is designed to detect and classify multiple objects in images or videos in a single evaluation. Unlike traditional methods such as R-CNN, YOLO performs detection in one step, making it significantly faster and suitable for real-time applications.

5.2 Role of YOLO in Object Detection

YOLO plays an important role in object detection by:

- Providing real-time detection of multiple objects in images and videos
- Dividing the input image into grid cells and predicting objects within each cell
- Simultaneously predicting bounding boxes and class probabilities
- Being widely used in applications such as surveillance systems, autonomous driving.

YOLO is popular because it achieves a good balance between speed and accuracy.

5.3 Key Features Utilized

During the internship, YOLO (You Only Look Once) was utilized for real-time object detection and tracking applications. The key features of YOLO that were used include:

- **Real-Time Object Detection:** YOLO enables fast detection of multiple objects in images and video streams, making it suitable for real-time surveillance and monitoring applications.
- **Accurate Object Localization:** YOLO identifies the exact position of detected objects using bounding boxes, which helps in determining object location and size within a frame.
- **Multi-Class Detection:** YOLO can detect and classify multiple object categories simultaneously, allowing efficient identification of different objects in a single image or video frame.
- **High-Speed Processing:** YOLO performs detection in a single neural network pass, providing faster processing compared to traditional object detection methods.

5.4 Object Detection and Tracking using YOLO

Coding:

```
import cv2
from ultralytics import YOLO
import os
```

```

# 1. Load YOLO model
# -----
model = YOLO("yolov8n.pt")

# -----
# 2. Input video
# -----
video_path = r"video.mp4"

if not os.path.exists(video_path):
    print(f"Video not found: {video_path}")
    exit()

cap = cv2.VideoCapture(video_path)

if not cap.isOpened():
    print("Error: Cannot open video.")
    exit()

# -----
# 3. Video properties
# -----
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = cap.get(cv2.CAP_PROP_FPS)

if fps == 0:
    fps = 30

# -----
# 4. Output video writer
# -----
output_path = r"output_tracked.mp4"

fourcc = cv2.VideoWriter_fourcc(*"mp4v")

out = cv2.VideoWriter(
    output_path,
    fourcc,
    fps,
    (width, height)
)

if not out.isOpened():
    print("Error: Cannot create output video.")
    exit()

# -----
# 5. Run tracking
# -----
results = model.track(
    source=video_path,
    stream=True,

```

```

        persist=True,
        classes=[0],    # person only
        show=False
    )

# -----
# 6. Process frames
# -----
for r in results:

    frame = r.orig_img

    if r.bboxes is not None:

        for box in r.bboxes:

            x1, y1, x2, y2 = map(int, box.xyxy[0])

            conf = float(box.conf[0])

            if box.id is not None:
                track_id = int(box.id[0])
            else:
                track_id = -1

            # Green bounding box
            cv2.rectangle(
                frame,
                (x1, y1),
                (x2, y2),
                (0, 255, 0),
                2
            )

            label = f"ID {track_id} | {conf:.2f}"

            cv2.putText(
                frame,
                label,
                (x1, y1 - 10),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.6,
                (0, 255, 0),
                2
            )

        out.write(frame)

    cv2.imshow("YOLO Person Tracking", frame)

    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

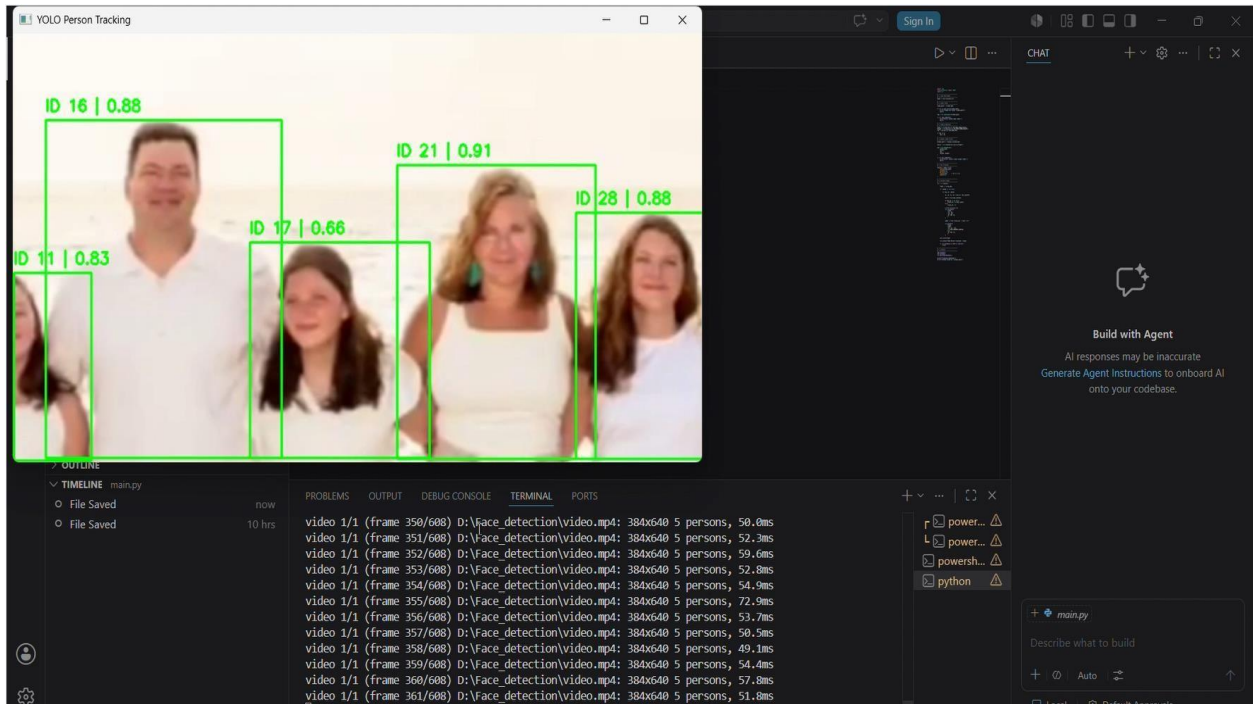
# -----

```

```
# 7. Cleanup
# -----
cap.release()
out.release()
cv2.destroyAllWindows()

print(f"Tracking completed.")

print(f"Output saved as: {output_path}")
```



https://drive.google.com/file/d/1oHZjsJzAxWkfvspq39kVxO2rM2xC5Ho7/view?usp=drive_link

5.5 Summary

The Face Detection Using Computer Vision project successfully demonstrated how artificial intelligence can enable machines to detect human faces accurately in real time using Python and OpenCV. This project provided practical exposure to image processing, object detection, and real-time application development, helping me strengthen my programming, analytical, and problem-solving skills. It also showed the growing importance of computer vision in modern applications such as security systems, attendance management, mobile authentication, and smart surveillance. Overall, the internship project effectively bridged the gap between theoretical learning and practical implementation while building a strong foundation for future work in Artificial Intelligence, Machine Learning, and Computer Vision.

6. HAAR CASCADE, DNN, MTCNN, AND ARCFACE – FACE DETECTION AND RECOGNITION

6.1 Introduction

Face detection and recognition are important areas of computer vision used to identify and verify human faces in images and videos. Various algorithms such as Haar Cascade, Deep Neural Networks (DNN), MTCNN, and ArcFace are widely used for accurate and efficient face analysis. These methods help machines detect facial regions and recognize individuals based on facial features.

6.2 Role of Face Detection Models in Computer Vision

Face detection models play a key role in many real-world applications. They help in locating human faces in images or video frames and preparing them for recognition tasks.

- Detect faces in images and live video streams
- Improve security systems like surveillance and authentication
- Enable facial recognition systems in mobile phones and applications
- Support AI-based applications such as attendance systems and emotion detection

These models form the foundation of face recognition systems.

1. HAAR CASCADE (OPENCV)

Haar Cascade uses **Haar-like features** (light and dark regions) to detect faces.

Example:

- Eyes are usually darker.
- Cheeks are usually brighter.

The model learns these patterns and searches for them in an image.

Working

1. Convert image to grayscale.
2. Extract Haar features.
3. Apply AdaBoost classifier.
4. Pass through Cascade stages.
5. Detect face.

2. OPENCV DNN FACE DETECTOR (DNN = Deep Neural Network)

Uses a **CNN (Convolutional Neural Network)** model inside OpenCV.

CNN automatically learns facial features like:

- Eyes
- Nose
- Mouth
- Face shape

Working

1. Input image.
2. CNN extracts features.
3. Predict face probability.
4. Draw bounding box.

3. MTCNN (Multi-task Cascaded Convolutional Neural Network)

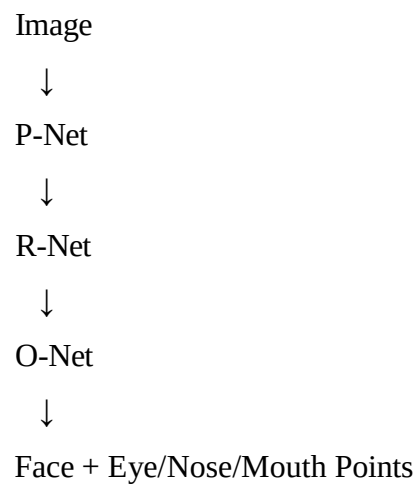
Uses 3 CNN networks:

P-Net : Finds possible face regions.

R-Net: Removes false detections.

O-Net: Produces final face box and landmarks.

Working

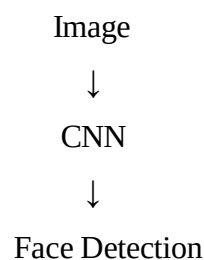


4. RetinaFace

Uses:

- CNN Backbone
- Feature Pyramid Network (FPN)

Working



6.3 Comparison of Haar Cascade, DNN, MTCNN, and ArcFace

Method	Type	Purpose	Accuracy	Speed	Key Feature
Haar Cascade	Machine Learning	Face detection	Low to Medium	Very Fast	Uses Haar-like features and cascade classifiers
DNN (Deep Neural Network)	Deep Learning	Face detection	High	Moderate	More robust detection in real-world conditions
MTCNN	Deep Learning	Face detection + landmark detection	Very High	Moderate to Slow	Detects faces and facial key points (eyes, nose, mouth)
ArcFace	Deep Learning	Face recognition	Very High	Moderate	Uses angular margin loss for better face feature separation

6.4. Applications in Face Detection and Recognition

Face detection and recognition technologies are widely used in various real-world applications to improve security, automation, and user convenience. Some of the major applications are:

- **Security Systems** – Face detection and recognition are used in surveillance cameras, airports, offices, and public places to identify individuals, monitor activities, and control access to restricted areas. These systems help improve public safety and reduce unauthorized entry.
- **Mobile Authentication** – Modern smartphones use facial recognition to unlock devices securely. This technology provides a fast, contactless, and convenient authentication method while protecting user data from unauthorized access.
- **Attendance Systems** – Educational institutions and organizations use face recognition to automatically record attendance. It eliminates manual attendance processes, prevents proxy attendance, and saves time by accurately identifying individuals.
- **Banking and Finance** – Banks and financial institutions use facial recognition for customer identity verification during account creation, online banking, ATM transactions, and digital payments. This enhances security and helps prevent identity fraud.

7. PROJECT WORK – FACE TRACKING AND ATTENDANCE MONITORING SYSTEM

The Face Tracking Attendance System is a Computer Vision-based solution designed to automatically detect, track, and recognize individuals from video streams while monitoring their movement within a predefined geofenced region. The system integrates object detection, face recognition, tracking, geofence monitoring, and event logging into a single pipeline. When a recognized person enters or exits the monitored region, attendance events are generated and stored in a CSV file.

1. Objectives

The main objectives of this project are:

- Detect people from video streams.
- Track individuals across multiple frames.
- Recognize known persons using face embeddings.
- Monitor entry and exit events within a region of interest (ROI).
- Generate automated attendance logs.
- Support smart attendance and surveillance applications.

2. System Architecture

The system follows a complete processing pipeline:

Video Input → YOLOv8 Detection & Tracking → InsightFace Face Detection → ArcFace Recognition → Face-Person Matching → Geofence Monitoring → FSM Processing → Attendance Logging (CSV)

3. Technologies Used

- Programming Language: Python
- Libraries: OpenCV, NumPy, Ultralytics YOLO, InsightFace, CSV, Datetime
- Development Tools: Visual Studio Code, Python Virtual Environment

4. Models Used

YOLOv8 (Person Detection & Tracking)

Used for detecting and tracking multiple persons in video frames. It assigns unique IDs and maintains tracking across frames using ByteTrack.

InsightFace (Face Detection & Recognition)

Used for detecting faces and generating ArcFace embeddings for recognition. It converts facial features into high-dimensional vectors for matching

Embedding Database (embeddings.pkl)

Stores pre-generated face embeddings for registered users to enable fast comparison during recognition.

5. Dataset Structure

The dataset is organized with one folder per person:

```
dataset/  
→Person_1/(images)  
→ Person_2/(images)
```

ArcFace embeddings are generated from these images and stored in embeddings.pkl.

6. Face Recognition Process

- Detect faces using InsightFace
- Generate ArcFace embeddings
- Compare with stored embeddings
- Match based on similarity score
- Output recognized name or “Unknown”

7. Geofence Monitoring

A predefined Region of Interest (ROI) is used to monitor movement.

- Entry event: when a person enters the ROI
- Exit event: when a person leaves the ROI

8. Finite State Machine (FSM)

The system tracks movement states:

OUTSIDE → ENTERED → INSIDE → EXITED → OUTSIDE

This ensures proper tracking of entry and exit events.

9. Attendance Logging

All events are stored in a CSV file:

Format:

Timestamp | Track_ID | Person_Name | Event

Example:

2026-06-10 10:15:20 | 7 | Ganesh | ENTERED

10. Applications

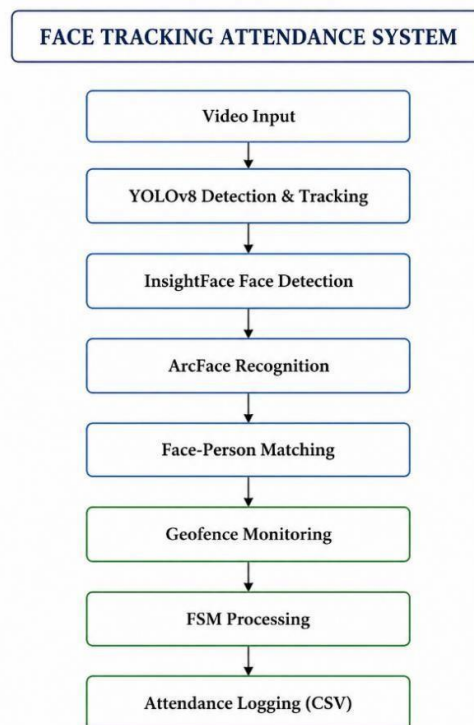
- Classroom attendance system
- Office entry tracking
- Smart building monitoring
- CCTV surveillance systems

11. Future Enhancements

- Real-time CCTV integration
- Multi-camera support
- Database integration (MySQL/PostgreSQL)
- Web dashboard for analytics
- Mobile notifications
- Face liveness detection

In conclusion, the Face Tracking Attendance System is an advanced computer vision-based solution that automates the process of attendance monitoring and management. By integrating face recognition, object detection, tracking algorithms, and geofence-based monitoring, the system can accurately identify individuals and record their movements within a specified area. The automatic generation of entry and exit attendance records helps eliminate manual errors and saves time compared to traditional attendance methods.

The system provides real-time monitoring capabilities, enhances security, and ensures efficient data management by storing attendance information in a structured CSV format. It can be further improved by incorporating cloud storage, mobile notifications, and advanced AI models for better accuracy and scalability. Overall, this system demonstrates the effective application of artificial intelligence and computer vision technologies in developing a smart, reliable, and automated attendance management solution.



8. RESULTS AND OBESERVATIONS

The Face Tracking Attendance System was successfully implemented and tested using real-time video streams. The system effectively combined person detection, face recognition, object tracking, geofencing, and attendance management into a single automated solution. During testing, the YOLOv8 model accurately detected multiple individuals in each video frame and maintained consistent tracking IDs using the ByteTrack algorithm. This ensured smooth tracking even when people were moving within the camera's field of view.

InsightFace with ArcFace embeddings provided reliable face recognition by comparing the detected faces with the registered face database. The system was able to recognize authorized individuals with high accuracy under normal lighting conditions while assigning an "Unknown" label to unregistered persons. This improved the reliability and security of the attendance process.

The geofence monitoring module accurately identified whether a tracked person was inside or outside the predefined Region of Interest (ROI). Entry and exit events were detected correctly, and the State Manager prevented duplicate attendance records by maintaining the status of each individual. This ensured that attendance was recorded only once for each valid entry, reducing errors caused by repeated detections.

All attendance events, including the person's name, date, time, and entry status, were successfully stored in a CSV file for future reference. The generated attendance records were accurate, organized, and easy to analyze. The system also displayed real-time detection results with bounding boxes, tracking IDs, confidence scores, and recognized names on the video output, making it easier to monitor the overall performance.

The experimental results demonstrated that the proposed system performs efficiently in real-time environments and significantly reduces manual effort in attendance management. The integration of YOLOv8, ByteTrack, ArcFace, and geofencing provided a robust, accurate, and automated solution suitable for educational institutions, offices, and other organizations requiring secure and contactless attendance monitoring.

The Face Tracking Attendance System successfully detected, tracked, and recognized individuals in real-time video streams with high accuracy. The YOLOv8 model effectively detected people, while ByteTrack maintained consistent tracking IDs across frames. InsightFace with ArcFace embeddings accurately recognized registered users and identified unknown individuals.

9. CHALLENGES AND SOLUTIONS

1. **Low Lighting and Image Quality Issues:** Poor lighting conditions and low-quality video frames reduced the accuracy of face detection and recognition, making it difficult to identify individuals consistently. The solution is to apply image preprocessing techniques and use robust deep learning models to improve detection under varying lighting conditions.
2. **Identity Mismatch During Tracking:** Tracking IDs occasionally switched between different individuals when they crossed paths or moved quickly. The solution is to use the ByteTrack algorithm with YOLOv8 to maintain consistent tracking identities across consecutive video frames.
3. **Face Recognition Accuracy Variation:** Similar-looking faces and changes in facial expressions occasionally caused incorrect recognition results. The solution is to use ArcFace embeddings, which provide more discriminative facial features and improve recognition accuracy.
4. **Occlusion of Faces:** Faces were sometimes partially covered by masks, hands, or other objects, making recognition more challenging. The solution is to use advanced face recognition models capable of identifying faces even with partial facial visibility.
5. **Multiple People in the Same Frame:** Simultaneously detecting and tracking several individuals increased the complexity of maintaining correct identities. The solution is to use a reliable multi-object tracking algorithm that accurately associates detections across successive frames.
6. **Real-Time Performance Delay:** Processing every video frame for detection, tracking, and recognition introduced slight delays during execution. The solution is to optimize the processing pipeline by reducing unnecessary computations and improving system efficiency.
7. **Integration of Multiple Modules:** Integrating person detection, face recognition, tracking, geofencing, and attendance logging into a single system required careful coordination. The solution is to develop the application using a modular architecture that enables smooth communication between all system components.

10. KEY LEARNINGS AND REFLECTIONS

The internship provided me with valuable practical experience in Computer Vision and Artificial Intelligence. I gained hands-on knowledge of image processing, face detection, face recognition, object detection, object tracking, and real-time video analytics. I learned to work with tools and technologies such as Python, OpenCV, YOLOv8, InsightFace, ArcFace, and ByteTrack while developing a Face Tracking Attendance System.

Through this internship, I strengthened my understanding of image preprocessing, deep learning-based face recognition, geofencing, attendance automation, and system integration. I also improved my programming, debugging, testing, and problem-solving skills by overcoming challenges encountered during project development. Working on a real-time application helped me understand how multiple AI models can be integrated into a complete and efficient system.

Overall, this internship enhanced both my technical knowledge and practical skills. It increased my confidence in developing real-world AI applications and provided valuable industry exposure. The experience motivated me to further explore advanced topics in Computer Vision, Deep Learning, and Artificial Intelligence for future academic and professional growth.

This project helped me gain practical experience in Computer Vision and real-time AI systems. I learned how to integrate multiple technologies such as YOLOv8, OpenCV, and InsightFace into a single working pipeline.

I improved my understanding of:

- Object detection and tracking techniques
- Face recognition using deep learning models
- Geofencing and event-based monitoring systems
- Data logging and system integration

This project strengthened my problem-solving skills and gave me hands-on experience in building real-world AI applications.

11. CONCLUSION

The Face Tracking Attendance System was developed to automate attendance using computer vision techniques. The system integrates **OpenCV** for video capture and processing, **YOLO** for real-time person detection and tracking, and advanced **face detection and recognition models** for identifying individuals accurately. These technologies work together to create a complete automated attendance pipeline.

The system successfully detects and tracks individuals in video streams and recognizes registered users based on facial features. It also monitors movement using geofence logic to generate entry and exit events, which are automatically stored as attendance records. This reduces the need for manual attendance marking and improves efficiency.

Overall, the project demonstrates how modern AI technologies such as OpenCV, YOLO, and face recognition models can be effectively combined to build a real-time, intelligent, and automated attendance management system suitable for classrooms and offices.

In addition, the use of deep learning-based models improves the overall robustness of the system, even in varying lighting conditions and different camera angles. This makes the solution more practical for real-world deployment.

The project also helped in understanding how multiple computer vision modules can be integrated into a single pipeline. It highlights the importance of combining detection, tracking, and recognition for building intelligent systems.

APPENDIX

Internship Completion Certificate

Completed a Computer Vision internship, covering OpenCV, YOLO, and face recognition models, with hands-on experience in object detection, tracking, and real-time attendance system development.

ANDUM TECHNOLOGIES PRIVATE LIMITED

12/660, B2, F3, PATHANJALI GARDENS, 7TH STREET, VEERABADRA
NAGAR, MEDAVAKKAM, CHENNAI, TAMILNADU, INDIA, 600100



Date : 14 Jun 2026

INTERNSHIP-2026/IT-004

TO WHOMSOEVER IT MAY CONCERN

This is to certify that **R.GANESH BABU**, student of St. Xavier's College (Autonomous), Palayamkottai has successfully completed his Internship on Computer Vision at our company Andum Technologies Private Limited during the period from 14th May 2026 to 14th June 2026. He has been a keen learner. His skills and efforts during the training were commendable and we wish him all the best for his future endeavours.

For Andum Technologies Private Limited

Sincerely

A handwritten signature in blue ink, appearing to be 'R. Ganesh Babu', is written over a light blue rectangular background.

Director

